

Ownership & Borrowing — Từ Bản Chất Đến Nâng Cao

File này đi từ vấn đề căn bản (vì sao Rust ra đời?), qua quy tắc ownership/borrowing, vào sâu cách borrow checker hoạt động, kết nối liên tục với **memory model** (xem [memory-model.md](#) và [memory-model-visual.md](#)).

Mục lục

Tầng 1 — Vì sao ownership tồn tại?

1. Ba căn bệnh muôn thuở của bộ nhớ
2. Hai trường phái giải quyết: manual vs GC
3. Triết lý Rust: ownership là gì?

Tầng 2 — Ownership cơ bản 4. Ba quy tắc vàng 5. Move — bản chất là gì? 6. Copy — khi nào và vì sao? 7. Clone — bản sao thật sự 8. Drop — hủy giá trị tự động

Tầng 3 — Borrowing 9. Vì sao phải borrow? 10. Hai loại reference: &T và &mut T 11. Quy tắc borrowing — vì sao tồn tại? 12. Reborrow — vay từ vay

Tầng 4 — Lifetime 13. Lifetime là gì — bản chất 14. Lifetime elision — đường tắt của compiler 15. 'static — sống mãi mãi 16. Lifetime trong struct & function

Tầng 5 — Bên trong Borrow Checker 17. Borrow checker hoạt động ra sao? 18. NLL — Non-Lexical Lifetimes 19. Polonius — thể hệ tiếp theo 20. Two-phase borrows

Tầng 6 — Interior Mutability 21. Bài toán: mutate qua &T 22. Cell, RefCell — single-thread 23. Mutex, RwLock — multi-thread 24. UnsafeCell — gốc rễ

Tầng 7 — Shared Ownership 25. Box — owner duy nhất trên heap 26. Rc / Arc — đếm tham chiếu 27. Weak — đếm không sở hữu

Tầng 8 — Liên hệ Memory Model 28. Move = zero-cost vì sao? 29. Borrowing & Cache friendliness 30. Drop deterministic = không GC pause 31. Aliasing — vũ khí tối ưu hóa

Tầng 9 — Patterns nâng cao 32. Type-state pattern 33. PhantomData — lifetime ảo 34. Self-referential structs — vì sao khó 35. Pin — neo cố định

Tầng 10 — Bẫy thường gặp 36. Common errors & cách đọc thông báo

TẦNG 1 — VÌ SAO OWNERSHIP TỒN TẠI?

1. Ba căn bệnh muôn thuở

Trước khi hiểu ownership, phải biết Rust đang **chữa** bệnh gì. Trong C/C++, có 3 loại bug ám ảnh lập trình viên suốt 50 năm:

Bệnh 1: Use-After-Free (UAF)

```
char* ptr = malloc(100);
free(ptr);
printf("%s\n", ptr);    // ❌ Đọc bộ nhớ đã giải phóng
```

Hậu quả: undefined behavior — có thể đọc rác, có thể đọc data của user khác (security hole!).

Memory model:

malloc(100): HEAP cấp 1 block 100 byte

data

← ptr

free(ptr): HEAP đánh dấu block này TRỐNG

FREE

← ptr vẫn trỏ vào đây!

malloc(100) lần 2: Allocator cấp lại block này cho biến khác

data của X

← ptr cũ VẪN trỏ vào!

→ Đọc ptr = đọc data của X (sai!)

Bệnh 2: Double Free

```
free(ptr);
free(ptr);    // ❌ Giải phóng 2 lần
```

Hậu quả: phá vỡ cấu trúc free list của allocator → crash hoặc bị khai thác bảo mật.

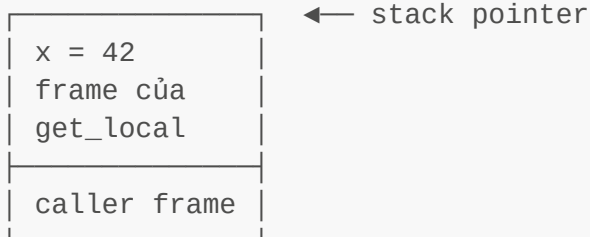
Bệnh 3: Dangling Pointer (con trỏ treo)

```
int* get_local() {
    int x = 42;
    return &x;    // ❌ Trả về địa chỉ biến local
}
int* p = get_local(); // p trỏ vào stack frame ĐÃ POP
```

Memory model:

Khi `get_local()` chạy:

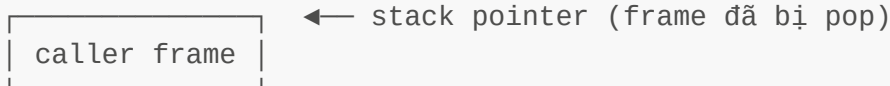
STACK:



p sẽ trỏ vào x

Khi `get_local()` return:

STACK:



p VẪN trỏ vào địa chỉ cũ → vùng nhớ rác / frame mới

Bệnh 4: Memory Leak (rò rỉ)

```
char* ptr = malloc(100);
// ... quên free() ...
```

Khác 3 cái trên — không gây crash ngay, nhưng app chạy lâu sẽ ăn hết RAM.

Bệnh 5: Data Race (đa luồng)

```
int counter = 0;
// Thread 1: counter++;
// Thread 2: counter++;
// → counter có thể = 1 thay vì 2!
```

(Đã giải thích chi tiết trong [memory-model.md](#).)

→ **5 bệnh này** chính là lý do Rust ra đời.

2. Hai trường phái giải quyết

Trước Rust, đã có 2 cách giải quyết, đều có nhược điểm:

Trường phái 1: Quản lý thủ công (C/C++)

Lập trình viên TỰ chịu trách nhiệm:
- malloc khi nào

- free khi nào
- Đảm bảo không 2 lần
- Đảm bảo không quên
- Đảm bảo thread-safe

ƯU: Nhanh nhất, control tuyệt đối

NHƯỢC: Nỗi người ≠ máy → bug thường xuyên
50 năm vẫn không giải quyết hết

Trường phái 2: Garbage Collector (Java/Go/Python/JS)

RUNTIME có 1 "người dọn rác":

- Định kỳ quét toàn bộ object
- Đánh dấu object còn được tham chiếu
- Free object không tham chiếu

ƯU: An toàn – lập trình viên không phải nghĩ

NHƯỢC:

- GC pause (STW – Stop The World): app tạm ngưng vài ms-vài giây
- Tốn RAM: GC cần ~2x bộ nhớ thực dùng
- Không deterministic: không biết khi nào object bị free
- Không phù hợp embedded / kernel / real-time

So sánh trực quan

	Manual (C)	GC (Java)	Rust
Tốc độ:	\$\$\$	\$	\$\$\$
An toàn mem:	✗	✓	✓
Predictable:	✓	✗ (GC pause)	✓
RAM overhead:	0	~2x	0
Đa luồng:	✗	△ (data race)	✓
Học khó:	△ (chia mảnh sang head)	★ (dễ)	★★★ (khó)
Embedded:	✓	✗	✓

3. Triết lý Rust

Câu hỏi: Có thể có "Manual nhưng an toàn" không?

Rust trả lời: **Có, nếu compiler kiểm tra giúp bạn lúc compile.**

Cách C:

Manual + Trust the programmer (BAD)

↓

Runtime: bug, crash, security hole

Cách Java:
 Auto + Pay runtime cost (BAD)
 ↓
 Runtime: GC pause, RAM bloat

Cách Rust:
 Manual + Compiler enforce rules (GOOD!)
 ↓
 Compile time: lỗi nếu vi phạm
 Runtime: nhanh như C, an toàn như Java

"Ownership" là gì — định nghĩa cốt lõi

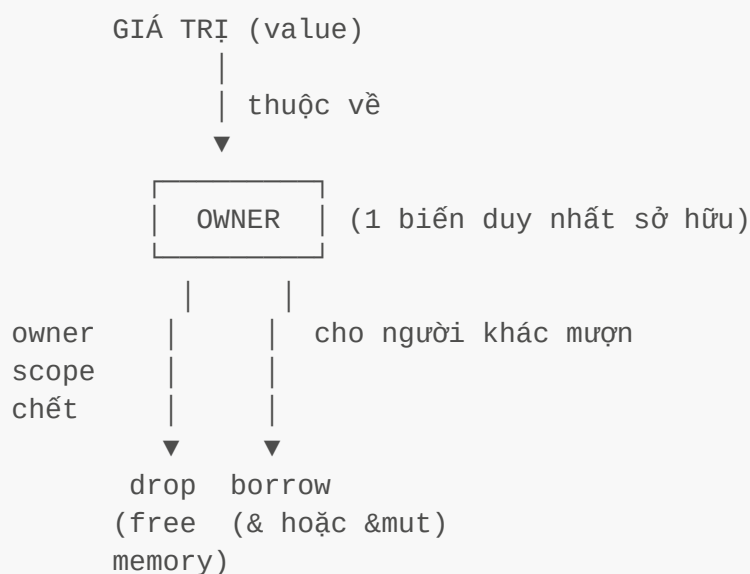
Ownership là hệ thống quy tắc do COMPILER kiểm tra, đảm bảo: với mỗi giá trị trên bộ nhớ, luôn có chính xác 1 người chịu trách nhiệm giải phóng nó.

Hệ quả:

- **Không UAF**: chỉ owner mới được dùng → khi owner chết, không ai khác còn tham chiếu hợp lệ
- **Không double-free**: chỉ 1 owner → chỉ free 1 lần
- **Không memory leak** (trong code an toàn): owner đi khỏi scope → tự free
- **Không dangling**: borrow checker kiểm tra reference không sống lâu hơn owner
- **Không data race**: chỉ 1 mutable reference → không 2 thread cùng ghi

→ Tất cả ràng buộc → **compile time**. Runtime giống C.

Sơ đồ tư duy



TẦNG 2 — OWNERSHIP CƠ BẢN

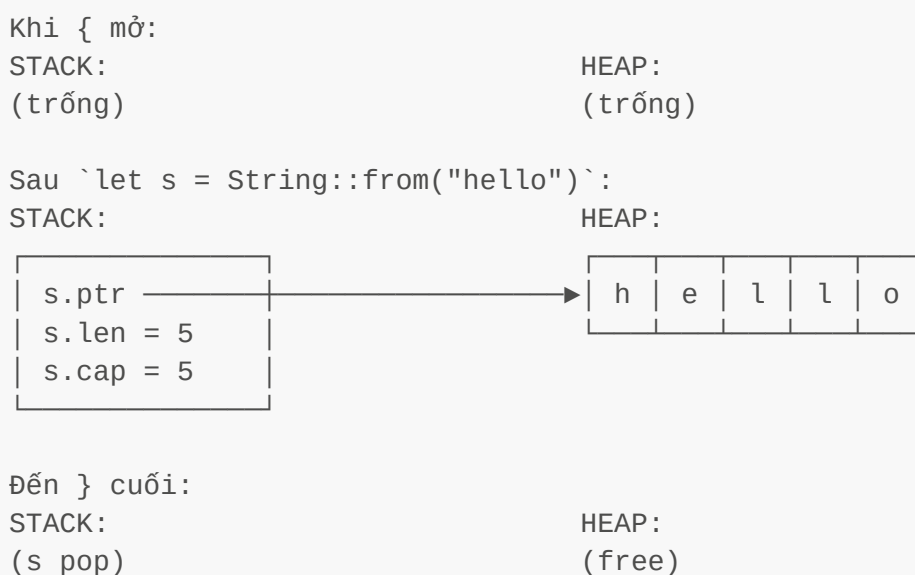
4. Ba quy tắc vàng

QUY TẮC 1: Mỗi giá trị có ĐÚNG 1 owner
 QUY TẮC 2: Khi owner đi khỏi scope, giá trị bị drop
 QUY TẮC 3: Có thể CHUYỂN ownership (move) sang biến khác

Ví dụ rất nhỏ

```
fn main() {
    let s = String::from("hello"); // s là owner
    // ...
} // ← Hết scope, s bị drop → heap data "hello" được free
```

Sơ đồ memory:



→ Không cần `free()`. Compiler **chèn drop tự động** tại `}`.

Cơ chế: RAII

Rust mượn từ C++: **R**esource **A**cquisition **I**s **I**nitialization.

Resource (heap memory, file, lock, socket...)
 ↓
 Gắn liền vào tuổi thọ của 1 biến local
 ↓
 Biến local hết scope → destructor (drop) chạy

↓
Resource được giải phóng

→ "Quên giải phóng" gần như **không thể** xảy ra.

5. Move

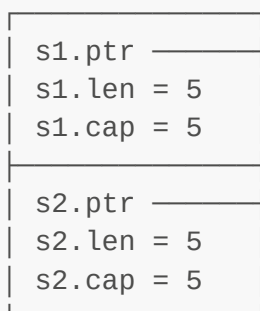
Đây là điểm gây bối rối nhất cho người mới.

Quy tắc: gán biến cho biến khác = MOVE

```
let s1 = String::from("hello");
let s2 = s1;                                // s1 MOVE sang s2
println!("{}", s1);                          // ❌ ERROR: s1 đã bị move
```

Vì sao lỗi? Vì nếu cho phép cả s1 và s2 tồn tại:

STACK:



HEAP:

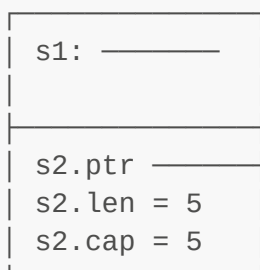


- HAI biến cùng trỏ vào 1 heap data
- Khi cả 2 hết scope, drop chạy 2 LẦN trên cùng heap → DOUBLE FREE!

Rust giải quyết: chỉ 1 owner

```
let s2 = s1;
```

STACK:



Quan trọng: move **KHÔNG** copy heap data. Chỉ copy 3 word (ptr, len, cap) trên stack. Heap data giữ nguyên — chỉ "đổi chủ".

Move xảy ra ở đâu?

```
// 1. Gán biến
let s2 = s1;

// 2. Truyền vào hàm
fn take(s: String) { ... }
take(s1);           // s1 move vào s

// 3. Trả về từ hàm
fn make() -> String {
    let s = String::from("hi");
    s           // s move ra ngoài (return)
}

// 4. Đẩy vào collection
vec.push(s1);       // s1 move vào vec
```

Cảnh giác: move trong vòng lặp

```
let s = String::from("hi");
for _ in 0..3 {
    take(s);       // ❌ Vòng 2 lỗi: s đã move vào lần 1
}
```

Sửa: clone hoặc borrow (sẽ học sau).

6. Copy

Một số kiểu **không bị move** mà bị **copy** (sao chép bit-for-bit).

```
let x: i32 = 5;
let y = x;           // x được COPY sang y
println!("{}", x);   // ✓ OK, x vẫn dùng được
```

Khi nào kiểu là Copy?

Một type là **Copy** khi có thể **sao chép bằng cách copy từng bit trên stack**, KHÔNG cần touch heap.

Copy types:

NOT Copy:

i8..i128, u8..u128	String	(có heap data)
f32, f64	Vec<T>	(có heap data)
bool, char	Box<T>	(heap)
&T (shared reference)	&mut T	(exclusive – không Copy)
Tuple/Array/Struct CHỈ KHI	Rc<T>	(cần tăng count nếu copy)
tất cả field đều Copy	HashMap	(heap)

Vì sao &T là Copy mà &mut T không?

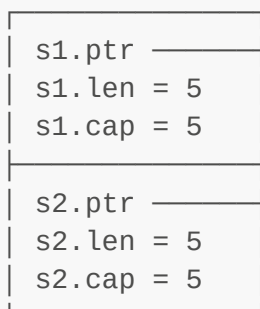
&T là 1 con trỏ (8 byte) → copy bit thì... vẫn cùng dữ liệu. Không sao.
→ Nhiều thread/nơi cùng đọc OK.

&mut T copy → 2 con trỏ ghi → có thể ghi cùng lúc → race!
→ Cấm copy &mut T.

Tại sao String không Copy mặc dù compiler "có thể" copy 3 word?

```
let s1 = String::from("hello");
let s2 = s1;          // GIẢ SỬ là copy
```

STACK:



HEAP:



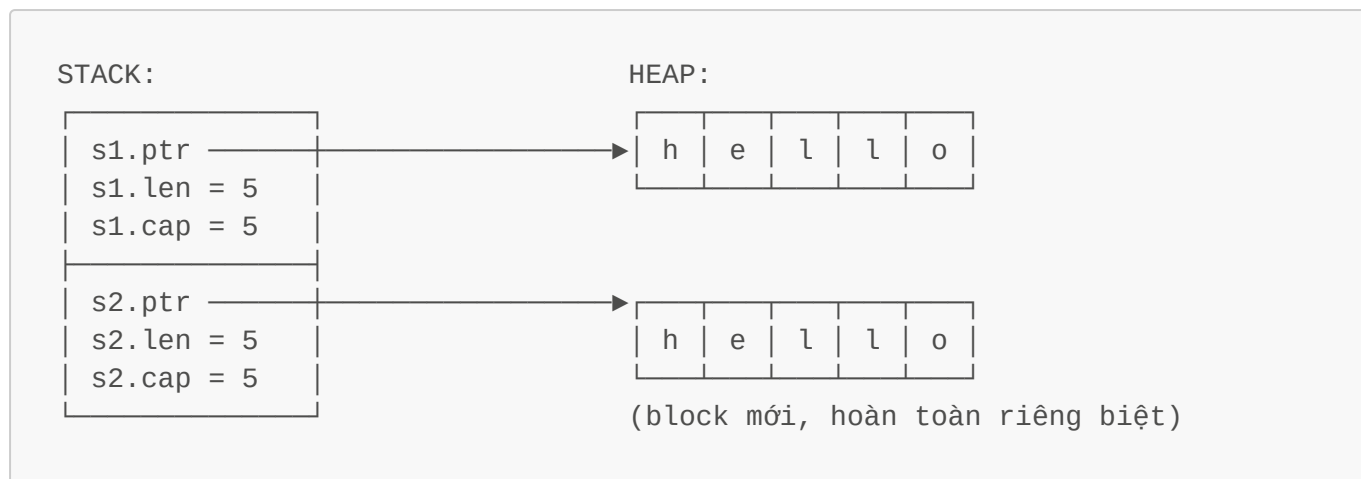
→ Hai chủ cùng heap → khi drop, đụng nhau → DOUBLE FREE!

→ Rust **chủ động cấm** Copy cho các kiểu có heap. Phải dùng **Clone** rõ ràng.

7. Clone

Clone = bản sao **thật sự**, tốn kém (deep copy).

```
let s1 = String::from("hello");
let s2 = s1.clone();           // Deep copy: cấp heap mới + copy bytes
println!("{}", s1, s2);       // ✓ OK
```



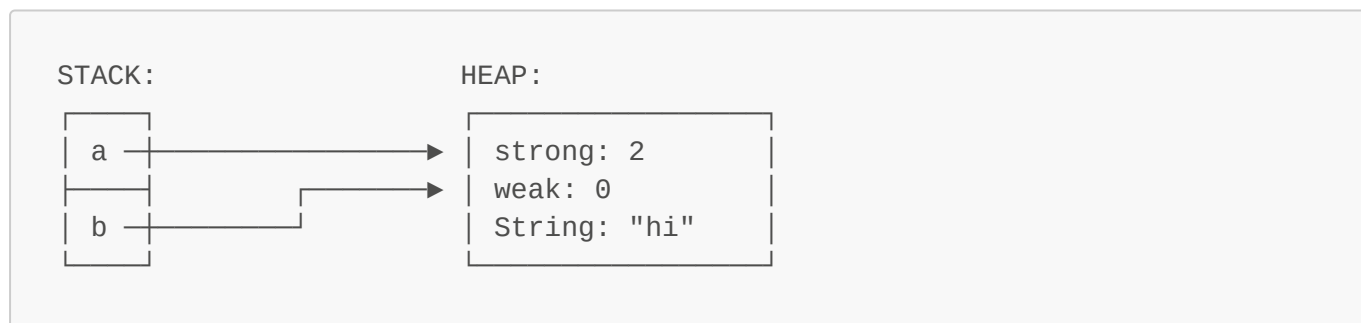
Triết lý: Clone phải **.clone()** rõ ràng

Copy: ngầm, vì rẻ (CPU register, vài byte)
Clone: tường minh, vì đắt (heap allocation)

→ Khi đọc code, thấy **.clone()** là biết "ô đây tốn kém, có muốn không?"

Rc/Arc clone — đặc biệt

```
let a: Rc<String> = Rc::new(String::from("hi"));
let b = Rc::clone(&a); // KHÔNG clone string, chỉ tăng counter
```



Đây là "shared ownership" — chi tiết ở Phần 26.

8. Drop

drop = "đốt sạch" 1 giá trị: giải phóng heap, đóng file, release lock...

Drop tự động tại cuối scope

```
fn main() {
    let s = String::from("hi");
    let v = vec![1, 2, 3];
}
```

```
// ... code ...
} // ← TỰ ĐỘNG: drop(v), drop(s) (NGƯỢC thứ tự khai báo)
```

Thứ tự drop: LIFO

```
let a = ...; ┌
let b = ...; ┤ — khai báo: a, b, c
let c = ...; └
}           — drop:    c, b, a (ngược lại!)
```

Vì sao ngược? Vì nếu **b** tham chiếu **a**, thì khi drop **a** trước, **b** sẽ trở vào rác.

```
let s = String::from("data");
let r = &s;           // r tham chiếu s
// drop order: r → s ✓
//           nếu drop s → r, thì khi drop s, r đang trỏ → r đã chết rồi
```

Drop trait tùy chỉnh

```
struct Connection {
    name: String,
}

impl Drop for Connection {
    fn drop(&mut self) {
        println!("Đóng kết nối {}", self.name);
    }
}

fn main() {
    let c = Connection { name: String::from("db") };
} // ← in: "Đóng kết nối db"
```

→ Đây là cách Mutex, File, TcpStream tự đóng — không cần `close()` thủ công.

Có thể drop sớm bằng `std::mem::drop`?

```
let s = String::from("hi");
drop(s);           // ← hủy ngay tại đây
println!("{}", s); // ✗ s đã chết
```

`std::mem::drop` không có gì đặc biệt — chỉ là 1 hàm `fn drop<T>(_: T) {}` nhận giá trị **bằng MOVE** → ownership chuyển vào, hàm kết thúc → drop bình thường.

TẦNG 3 — BORROWING

9. Vì sao phải borrow?

Move rất phiền nếu chỉ muốn **xem** không muốn **sở hữu**.

```
fn print_len(s: String) {
    println!("{}", s.len());
} // ← drop s (mất luôn)

fn main() {
    let s = String::from("hi");
    print_len(s);           // s move vào print_len
    println!("{}", s);      // ❌ s đã chết!
}
```

→ Phải clone? **Không!** clone tốn kém.

→ Phải **borrow** — vay không sở hữu.

```
fn print_len(s: &String) {      // & = vay
    println!("{}", s.len());
}

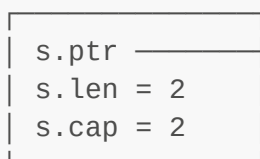
fn main() {
    let s = String::from("hi");
    print_len(&s);               // & = đưa địa chỉ, không move
    println!("{}", s);          // ✓ OK
}
```

Reference là gì — bản chất

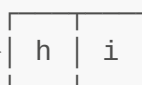
Reference là **CON TRỎ** an toàn, được borrow checker kiểm tra để **không bao giờ dangling**.

```
let s = String::from("hi");
```

STACK:

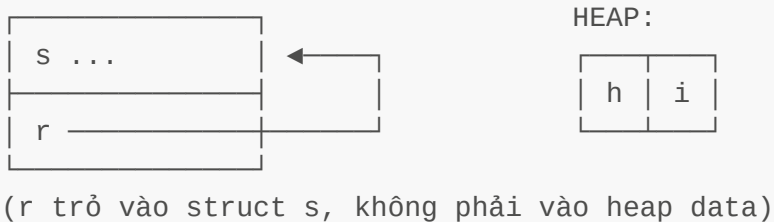


HEAP:



```
let r = &s;
```

STACK:



`&s` lúc compile = 8 byte pointer (địa chỉ của struct s trên stack).

10. Hai loại reference

`&T` — shared reference (immutable)

```
let s = String::from("hi");
let r1 = &s;           // shared borrow
let r2 = &s;           // có thể có NHIỀU shared borrow
println!("{}", s, r1, r2); // ✓
```

Đặc tính:

- Có thể có nhiều `&T` cùng lúc
- Không thể sửa qua `&T`
- Copy được (cheap)

`&mut T` — exclusive reference (mutable)

```
let mut s = String::from("hi");
let r = &mut s;
r.push_str(" world");
println!("{}", r);
```

Đặc tính:

- Chỉ có **MỘT** `&mut T` tại 1 thời điểm
- Không cùng tồn tại với `&T`
- Có thể sửa qua `&mut T`
- KHÔNG Copy

Quy tắc vàng

Tại **MỖI** thời điểm, mỗi giá trị có:

EITHER:

- 1 hoặc nhiều `&T` (chỉ đọc)

OR:

- đúng 1 `&mut T` (đọc + ghi, độc quyền)

KHÔNG BAO GIỜ:

- `&mut T + &T` cùng lúc
- 2 `&mut T` cùng lúc

11. Quy tắc borrowing — vì sao tồn tại?

Vì sao "nhiều `&T` cùng lúc" là OK?

Thread A: `r1 = &s`; đọc `s.len()`

Thread B: `r2 = &s`; đọc `s.len()`

→ Không ai sửa `s` → không có race → safe!

Về memory model:

CPU cache:

Core 0 đọc cache line chứa `s` → load vào L1 (shared state)

Core 1 đọc cùng cache line → load vào L1 (shared state)



Cả 2 cùng "Shared" state — OK

Vì sao "1 `&mut T`" là độc quyền?

Vì nếu 2 `&mut` tồn tại cùng lúc:

Thread A: `r1.push_str("a");` // sửa `s`

Thread B: `r2.push_str("b");` // sửa `s`

→ Race! Có thể corrupt heap.

Hoặc tệ hơn — vector resize giữa lúc đọc:

```
let mut v = vec![1, 2, 3];
let first = &v[0]; // &i32, trỏ vào heap
v.push(4); // có thể realloc heap → first dangling!
println!("{}", first); // ❌
```

Trong C++:

Vec ban đầu (cap = 3):

HEAP:

1	2	3
---	---	---

▲
first

push(4): cap không đủ → realloc đến block lớn hơn:

HEAP:

1	2	3	4	.	.	.
---	---	---	---	---	---	---

(block CŨ đã free!)

▲
first vẫn trỏ vào block cũ → DANGLING

Rust **cấm** tại compile time:

```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
```

```
-->
```

```
|
```

```
| let first = &v[0];
```

```
| ----- immutable borrow occurs here
```

```
| v.push(4);
```

```
| ^^^^^^^^^^^ mutable borrow occurs here
```

```
| println!("{}", first);
```

```
| ----- immutable borrow later used here
```

→ Quy tắc borrowing **chính là** mô hình hóa quy tắc memory safety thành rule có thể kiểm tra ở compile time.

Hai cách hình dung

Cách 1: Sách thư viện

&T = nhiều người mượn sách CHỈ ĐỌC (photo cũng được)

&mut T = 1 người mượn sách để VIẾT VÀO

Trong lúc viết, không ai khác được đọc/viết

Cách 2: Single-writer, multi-reader (kiến trúc)

Đây chính là pattern RwLock được nâng lên thành quy tắc ngôn ngữ.

RwLock kiểm tra runtime, Rust kiểm tra compile time.

12. Reborrow

Khi pass &mut sang hàm khác, không hẳn là move:

```
fn modify(s: &mut String) {
    s.push_str("!");
}

fn main() {
    let mut s = String::from("hi");
    let r = &mut s;
    modify(r);           // r được REBORROW, không move
    modify(r);           // ✓ vẫn dùng được r
}
```

Reborrow là gì?

`r: &mut String`

Khi gọi `modify(r)`:

Compiler tự động chèn: `modify(&mut *r);`

▲▲▲▲

reborrow:

- dereference `r` → `String`
- lấy `&mut` → tạo reference MỚI
- reference cũ (`r`) bị "đóng băng" tạm thời cho đến khi cái mới chết

Trước reborrow:
return:

`r` → `s`
lại)

Trong khi `modify` chạy:

`r` (BLOCKED)

`new_ref` → `s`
(đây là cái `modify` dùng)

`modify` return → `new_ref` drop

Sau khi `modify`

`r` → `s` (active

→ Sau khi `modify` xong, `r` "thức dậy". Đây là lý do code đa số "chỉ chạy" mà không cần ai dạy reborrow.

TẦNG 4 — LIFETIME

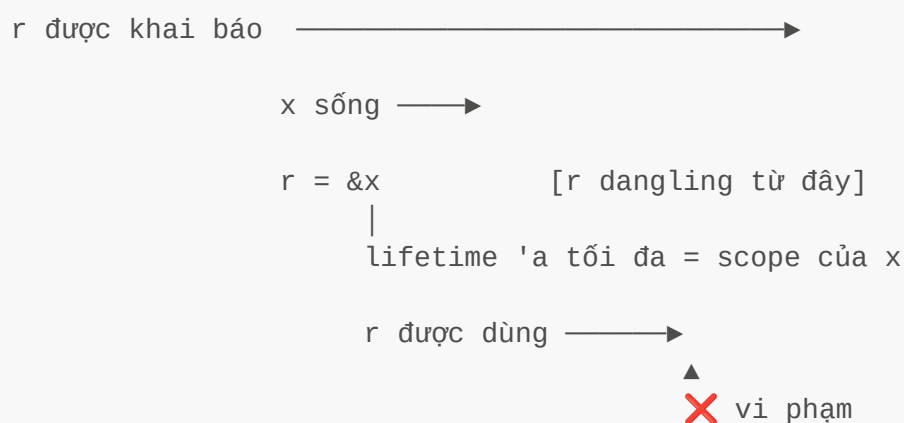
13. Lifetime là gì

Lifetime = khoảng thời gian (compile-time) mà reference được phép sống. Borrow checker dùng nó để chứng minh reference không dangling.

Trực quan: lifetime là tuổi thọ

```
let r;                // r: &i32, lifetime 'a bắt đầu
{
    let x = 5;        // x sống trong block trong
    r = &x;           // r mượn x → 'a = lifetime của x
}                    // x chết
println!("{}", r);    // ❌ r tham chiếu x đã chết
```

Timeline:



Lifetime trong signature

```
fn longest<'a>(s1: &'a str, s2: &'a str) -> &'a str {
    if s1.len() > s2.len() { s1 } else { s2 }
}
```

Đọc: "Cho 2 reference cùng lifetime 'a, trả về reference cũng lifetime 'a."

→ **Lifetime ở đây là contract**: caller bảo đảm s1 và s2 sống ít nhất 'a, callee bảo đảm trả về ref sống 'a.

Borrow checker dùng lifetime để làm gì?

Để **chứng minh**: reference đầu ra **không sống lâu hơn** dữ liệu nó tham chiếu.

```
let s1 = String::from("long");
let result;
{
    let s2 = String::from("short");
    result = longest(&s1, &s2); // 'a = min(s1, s2) = s2's scope
}
```

```
} // s2 chết
println!("{}", result); // ✗ result có thể trỏ s2
```

14. Lifetime elision

99% code không cần viết 'a — compiler tự suy ra.

3 quy tắc elision

QUY TẮC 1: Mỗi reference input đều có lifetime riêng

```
fn foo(x: &str, y: &str) {...}
↓ becomes ↓
fn foo<'a, 'b>(x: &'a str, y: &'b str) {...}
```

QUY TẮC 2: Nếu CHÍNH XÁC 1 input lifetime, output dùng lifetime đó

```
fn first(s: &str) -> &str {...}
↓ becomes ↓
fn first<'a>(s: &'a str) -> &'a str {...}
```

QUY TẮC 3: Nếu có &self hoặc &mut self, output dùng lifetime của self

```
impl S {
    fn name(&self, other: &str) -> &str {...}
}
↓ becomes ↓
fn name<'a, 'b>(&'a self, other: &'b str) -> &'a str {...}
```

Khi compiler không suy ra được → bạn phải tự viết 'a.

15. 'static

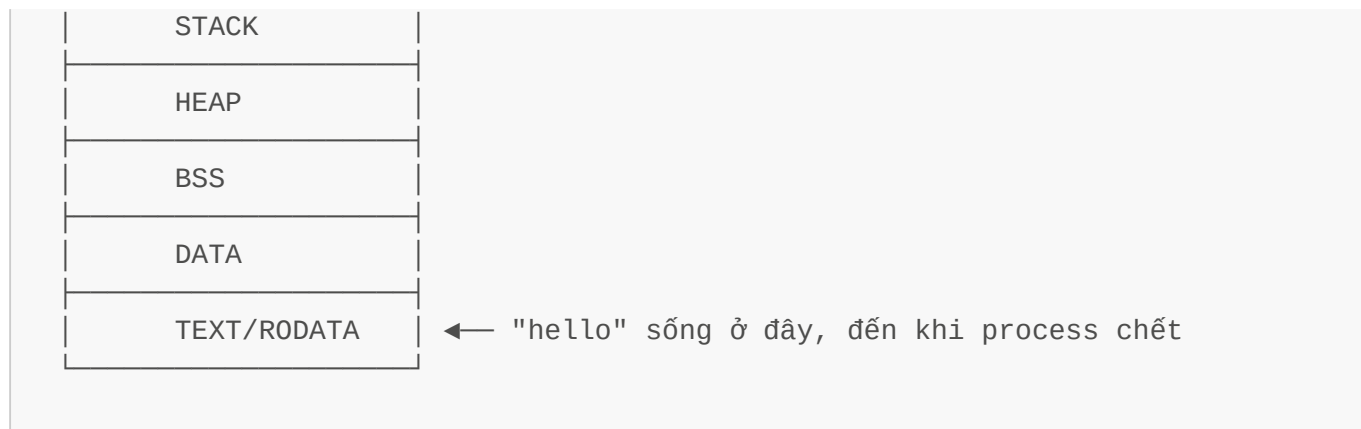
'static = sống mãi mãi (đến hết chương trình).

Hai loại 'static

1. String literal: nằm trong segment TEXT/RODATA của binary.

```
let s: &'static str = "hello"; // "hello" được nhúng vào binary
```

Process Address Space:



→ Không bao giờ bị free → an toàn lưu `&'static str` mãi.

2. T: 'static — owned values không chứa reference ngắn hơn

```
let s: String = String::from("hi");
// s: 'static (vì String tự sở hữu, không vay ai)

fn spawn<F: FnOnce() + Send + 'static>(f: F) { ... }
//
// closure không được "vay" gì có
lifetime ngắn
```

Cảnh giác: 'static không có nghĩa là "literal"

```
let s = String::from("hi");
// s là String – nó OWN heap data
// s: 'static? CÓ – vì kiểu String không chứa reference ngắn

let r: &str = &s; // r là &'a str với 'a = scope của s
// r: 'static? KHÔNG – vì r vay s, s chết khi hàm return
```

16. Lifetime trong struct & function

Struct chứa reference

```
struct Important<'a> {
    part: &'a str,
}
```

'a ở đây có nghĩa: "Instance của `Important` không được sống lâu hơn `part`."

```
let i;
{
```

```
let s = String::from("hi");
i = Important { part: &s };
}
println!("{}", i.part); // ❌ s đã chết
```

Ràng buộc nhiều lifetime

```
struct Parser<'a, 'b> {
    input: &'a str,
    output: &'b mut Vec<String>,
}
```

'a và 'b có thể khác nhau → parser có thể đọc input rất ngắn nhưng ghi vào output lâu dài.

Lifetime bounds

```
fn print<T: Debug>(x: T) where T: 'static {
    // x không được chứa reference ngắn hơn 'static
}
```

TẦNG 5 — BÊN TRONG BORROW CHECKER

17. Borrow checker hoạt động ra sao?

Đây là **compile-time** algorithm. Không có gì chạy lúc runtime.

Bước 1: Borrow checker phân tích từng "borrow"

Mỗi & tạo ra 1 borrow. Compiler ghi lại:

- Borrow của biến nào?
- Loại gì (shared/exclusive)?
- Sống từ đâu đến đâu?

```
let mut s = String::from("hi");
let r1 = &s;           // borrow #1: shared, &s, alive ?
let r2 = &s;           // borrow #2: shared, &s, alive ?
println!("{}", r1, r2); // r1, r2 dùng đây
                        // → borrow #1 và #2 alive đến đây
let r3 = &mut s;       // borrow #3: exclusive, &mut s, alive ?
r3.push_str("!");      // r3 dùng đây
```

Bước 2: Tính "alive region" của mỗi borrow

Trước NLL (Non-Lexical Lifetime), alive region = scope `{}`. Sau NLL = từ tạo đến **lần dùng cuối**.

```
let mut s = String::from("hi");
let r = &s;           // borrow start
println!("{}", r);    // borrow last use – alive region kết thúc Ở ĐÂY
s.push_str("!");      // ✓ OK với NLL (cũ thì lỗi)
```

Bước 3: Kiểm tra xung đột

```
For mỗi cặp borrow (A, B):
  IF A.alive_region ∩ B.alive_region ≠ ∅:
    IF A.kind == exclusive OR B.kind == exclusive:
      ERROR
```

Ví dụ phân tích

```
let mut v = vec![1, 2, 3];
let first = &v[0];
println!("{}", first);    // first dùng cuối
v.push(4);                // mượn v &mut
```

Borrow regions:

```
first (&i32):    [=====]
                 khai báo   println
```

```
push (&mut v):    [==]
                  .push()
```

Có chồng nhau? KHÔNG (NLL kết thúc first ngay sau println)
→ OK!

Nếu không NLL:

```
first:    [=====] (alive đến cuối block)
push:      [==]
```

Chồng nhau! ERROR.

18. NLL — Non-Lexical Lifetimes

Trước Rust 2018, lifetime gắn liền với **lexical scope** (cặp `{}`).

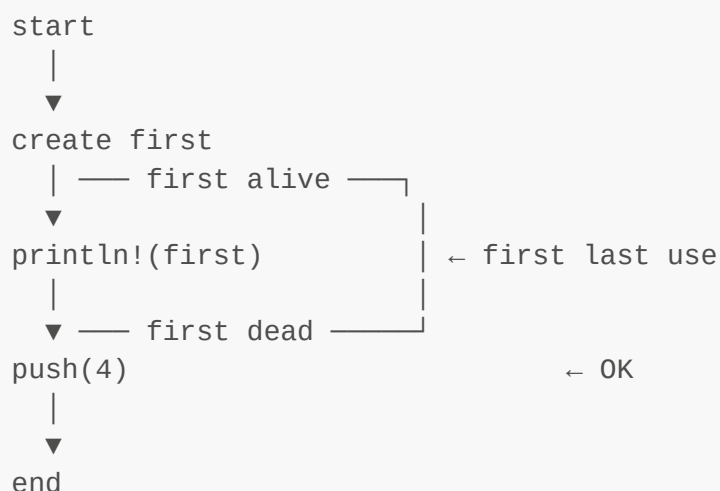
```
// Before NLL:
let mut v = vec![1, 2, 3];
let first = &v[0];
println!("{}", first);
v.push(4);           // ✗ ERROR before NLL (first alive đến ``)
```

NLL (từ 2018) tính alive region đến **last use thực sự**:

```
// After NLL:
let mut v = vec![1, 2, 3];
let first = &v[0];
println!("{}", first); // ← first last use
// (first đã chết, mặc dù scope chưa đóng)
v.push(4);             // ✓ OK
```

NLL là gì kỹ thuật

NLL phân tích flow control của program (Control Flow Graph) và tính chính xác **points** nào borrow còn alive.



19. Polonius — thế hệ tiếp theo

NLL vẫn không hoàn hảo. Có case Rust **rõ ràng an toàn** nhưng vẫn bị từ chối.

Ví dụ kinh điển: conditional return

```
fn get_or_insert(map: &mut HashMap<i32, String>, key: i32) -> &String {
    if let Some(v) = map.get(&key) {
        return v;           // ✗ lỗi với NLL
    }
}
```

```
map.insert(key, String::from("default"));
map.get(&key).unwrap()
}
```

NLL nghĩ rằng `map.get(&key)` borrow alive đến cuối hàm (vì có thể return), nhưng thực ra chỉ alive trong nhánh `if`.

Polonius

Polonius là borrow checker thế hệ mới (đang trong nightly), dùng **logic programming** (Datalog) để phân tích chính xác hơn.

Tư tưởng:

- Thay vì track "vùng" sống của borrow
- Track "fact": borrow nào xuất phát từ đâu, đi đến đâu, qua những path nào
- Dùng Datalog để suy luận

→ Khi Polonius stable, code trên sẽ compile thành công.

20. Two-phase borrows

Bài toán kỳ lạ:

```
let mut v = vec![1, 2, 3];
v.push(v.len()); // Hợp lý? Vâng. Compile được không?
```

Phân tích:

```
v.push(v.len())
  ▲      ▲
  |      |
  |      |
&mut v &v ← cùng tồn tại? Có vẻ vi phạm!
```

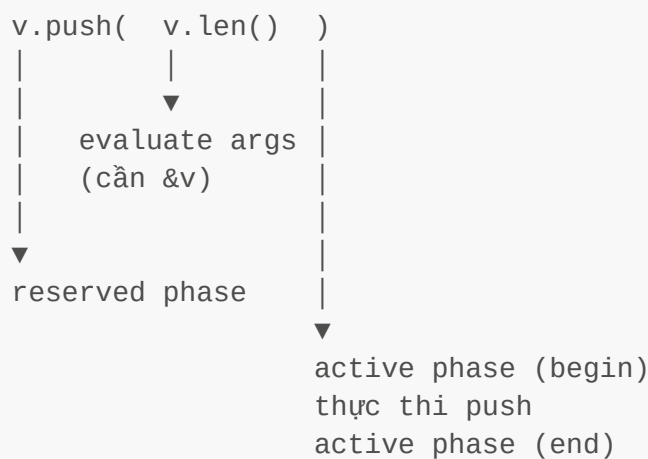
Nhưng Rust cho phép — vì **two-phase borrows**:

Borrow `&mut v` có 2 phase:

- Phase 1: "Reserved" — bắt đầu khi gọi `v.push(...)` chưa đọc quyền, chỉ "đặt chỗ"
- Phase 2: "Active" — bắt đầu khi push thực sự dùng `&mut self` đọc quyền từ đây

Giữa 2 phase, các &v khác (như `v.len()`) vẫn được phép!

Timeline:



→ Đây là 1 trong những "lý do bí mật" code Rust "chỉ chạy" mà bạn không cảm nhận quy tắc.

TẦNG 6 — INTERIOR MUTABILITY

21. Bài toán: mutate qua &T

Quy tắc: chỉ `&mut T` mới sửa được. Nhưng đôi khi:

- API yêu cầu &T (vd: `impl Hash for X { fn hash(&self, ...) }`)
- Cần "lazy initialize" (chỉ khởi tạo khi đọc lần đầu)
- Multiple owner (Rc) — tất cả chỉ có &T, không ai &mut được

→ Cần **interior mutability**: "sửa bên trong, qua &T".

Mô hình tư duy

Bình thường:

`&mut T` → có thể sửa

Interior Mutability:

`&T` → `Cell/RefCell/Mutex` → có thể sửa
 ↑
 "Vỏ" cho phép mutate bên trong
 dù từ ngoài chỉ thấy &T

22. Cell & RefCell

`Cell<T>` — đơn giản nhất (chỉ cho Copy)


```
use std::cell::Cell;

let c = Cell::new(5);
c.set(10);           // OK qua &Cell<i32>
println!("{}", c.get());
```

Cơ chế: `get/set` chỉ là **memcpy** (vì T: Copy). Không có pointer ra ngoài → không thể tạo dangling.

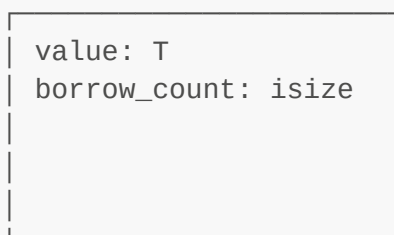
RefCell<T> — borrow runtime

```
use std::cell::RefCell;

let c = RefCell::new(vec![1, 2, 3]);
c.borrow_mut().push(4);           // tạo &mut RefMut tạm thời
let r = c.borrow();               // &Ref
println!("{:?}", r);
```

RefCell hoạt động ra sao

RefCell<T> internal:



← runtime counter:
 0 = không ai mượn
 >0 = N shared borrow
 -1 = 1 exclusive borrow

```
c.borrow():
  IF count >= 0: count++; trả về Ref<T>
  IF count == -1: PANIC!
```

```
c.borrow_mut():
  IF count == 0: count = -1; trả về RefMut<T>
  IF count != 0: PANIC!
```

Khi Ref/RefMut drop:
 Khôi phục count

→ Quy tắc borrow giống &T/&mut T, nhưng **kiểm tra runtime**. Vi phạm → panic.

Khi nào dùng RefCell?

- Bạn KNOW rằng borrow rules được tuân thủ, nhưng compiler không CHỨNG MINH ĐƯỢC.


```
read data → CHẮC = 42
m.unlock();
```

→ Mutex tự động cung cấp **happens-before** giữa unlock và lock kế tiếp. Không cần Atomic ordering thủ công.

RwLock

```
use std::sync::RwLock;

let lock = RwLock::new(vec![1, 2, 3]);

// Nhiều reader cùng lúc:
let r1 = lock.read().unwrap();
let r2 = lock.read().unwrap();    // ✓ OK

// Hoặc 1 writer độc quyền:
drop(r1); drop(r2);
let mut w = lock.write().unwrap();
w.push(4);
```

→ Pattern "multiple readers, single writer" như &T/&mut T, nhưng runtime.

24. UnsafeCell — gốc rễ

`UnsafeCell<T>` là **PRIMITIVE** duy nhất trong Rust thực sự cho phép mutate qua &T.

```
pub struct UnsafeCell<T> {
    value: T,
}
```

Hành vi đặc biệt: compiler **không** tối ưu hóa giả định rằng `&UnsafeCell<T>` là immutable.

```
Cell<T>      = UnsafeCell<T> + API copy-only an toàn
RefCell<T>   = UnsafeCell<T> + counter runtime
Mutex<T>     = UnsafeCell<T> + OS lock
Atomic<T>    = UnsafeCell<T> + atomic intrinsics
```

→ Tất cả interior mutability đều xây trên `UnsafeCell` ở dưới cùng.

Vì sao đặc biệt?

Compiler có thể giả định:

&T immutable → memoize giá trị, không phải đọc lại

Nhưng nếu $T = \text{UnsafeCell}\langle U \rangle$, compiler buộc đọc lại mỗi lần. Đây là contract đặc biệt giữa compiler và `UnsafeCell`.

→ Nếu bạn tự tay đảm bảo borrow rules, bạn có thể dùng `UnsafeCell` trong `unsafe` code để xây primitive mới.

TẦNG 7 — SHARED OWNERSHIP

25. Box — owner duy nhất trên heap

```
let b: Box<i32> = Box::new(42);
```



Box giải quyết vấn đề gì?

1. Kích thước không biết lúc compile

```
let arr: [i32; ?]; // ❌ ? — phải biết
let v: Vec<i32>;   // ✓ Vec ở heap, header 24B trên stack
let b: Box<[i32]>; // ✓ slice ở heap
```

2. Recursive types

```
enum List {
    Cons(i32, Box<List>), // Phải Box<List> chứ không List
    Nil,
}
```

Vì nếu không Box → `List` có size = $i32 + \text{size}(\text{List}) = i32 + \text{size}(\text{List})$ → vô hạn!

3. Trait object

```
let drawable: Box<dyn Draw> = Box::new(Circle { ... });
```

Sẽ giải thích trong "dynamic dispatch" — Box giúp lưu trait object kích thước không biết.

Box "siêu lực"

Box có ownership semantics y hệt String:

- Move khi gán
- Drop tự free heap
- $\&\text{Box} \approx \&\text{T}$ (auto deref)

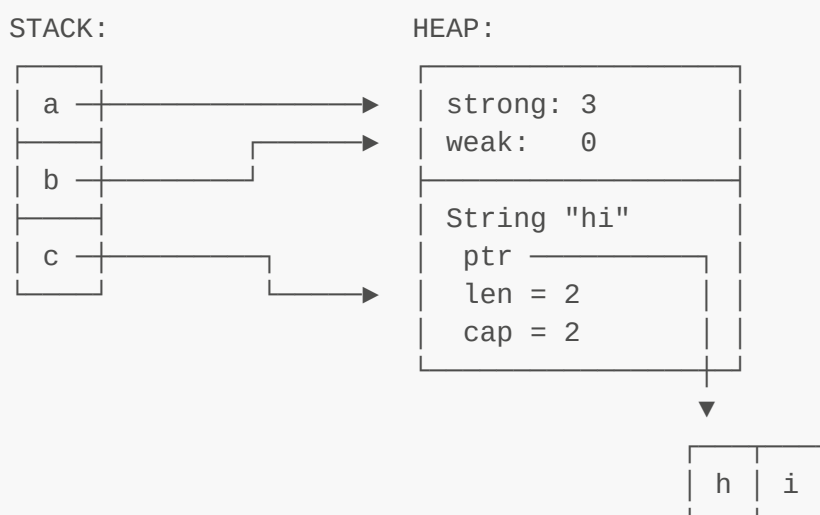
26. Rc & Arc

Rc — Reference Counted (single-thread)

```
use std::rc::Rc;

let a = Rc::new(String::from("hi"));
let b = Rc::clone(&a);
let c = Rc::clone(&a);

println!("count = {}", Rc::strong_count(&a)); // 3
```



Khi clone Rc:

```
Rc::clone(&a):
```

- atomic_increment? KHÔNG, chỉ ++ thông thường (single-thread)
- tạo Rc<T> mới trở vào cùng header

```
Khi Rc drop:
- count--
- IF count == 0: free heap (header + value)
```

Arc — Atomic Reference Counted

Y hệt Rc, nhưng counter dùng **atomic** → an toàn đa luồng.

```
use std::sync::Arc;

let data = Arc::new(vec![1, 2, 3]);
let data1 = Arc::clone(&data);
std::thread::spawn(move || {
    println!("{:?}", data1);
});
println!("{:?}", data);
```

Cost: atomic operations chậm hơn ~5-10x so với increment thường.

Khi nào dùng gì?

- 1 thread duy nhất → Rc (nhanh hơn)
- Đa luồng → Arc (an toàn)

Cảnh báo: Rc/Arc cho IMMUTABLE shared

```
let a = Rc::new(vec![1, 2, 3]);
let b = Rc::clone(&a);
a.push(4); // ❌ a là &Rc<Vec> không phải &mut
```

Vì nhiều owner nên không ai có &mut. Muốn sửa? Kết hợp với RefCell/Mutex:

```
let a = Rc::new(RefCell::new(vec![1, 2, 3]));
let b = Rc::clone(&a);
a.borrow_mut().push(4); // ✓
```

→ Pattern: `Rc<RefCell<T>>` (single-thread), `Arc<Mutex<T>>` (multi-thread).

27. Weak — đếm không sở hữu

`Weak<T>` = pointer **không** giữ object sống.

Bài toán: cycle reference

```
struct Node {
    children: Vec<Rc<Node>>,
    parent: Rc<Node>,      // ❌ cycle
}
```

parent → Node A → children → Node B
 |
 | parent
 ↓
 Node A ← cycle!

Node A.strong = 2 (root + B.parent)

Node B.strong = 1 (A.children)

Khi root drop: A.strong = 1, B vẫn alive

Nhưng A.children vẫn giữ B → B.strong = 1

B.parent giữ A → A.strong = 1

→ KHÔNG BAO GIỜ về 0 → MEMORY LEAK!

Giải pháp: Weak

```
struct Node {
    children: Vec<Rc<Node>>,
    parent: Weak<Node>,    // ✓ không tăng strong count
}
```

Weak chỉ tăng weak count.

Khi muốn truy cập: weak.upgrade() → Option<Rc<T>>

- Some(rc) nếu object còn sống
- None nếu đã bị drop

Node A: strong=1 (root), weak=1 (B.parent)

Node B: strong=1 (A.children)

Khi root drop:

A.strong = 0 → A bị drop

A bị drop → A.children drop → B.strong = 0 → B drop

✓ Không leak

TẦNG 8 — LIÊN HỆ MEMORY MODEL

28. Move = zero-cost vì sao?

Trên thực tế, sau khi compiler tối ưu, **move thường không tạo bất kỳ instruction nào**.

Ví dụ

```
fn make() -> String {
    String::from("hi")
}

fn main() {
    let s = make();
    println!("{}", s);
}
```

Bạn nghĩ: **make** tạo String trong frame nó, rồi copy về frame **main**?

Thực tế: compiler dùng **Return Value Optimization (RVO)**. String được **xây trực tiếp** trong frame của **main**.

TRƯỚC RVO (lý thuyết):

main frame:

```
s = ?
```

make frame:

```
temp = "hi"
```

| MOVE



main frame:

```
s = temp
```

SAU RVO (thực tế):

main frame:

```
s.ptr —> hp
s.len = 2
s.cap = 2
```

(make không có frame nào!)

→ Move = **chỉ là quy tắc compile-time**. Lúc runtime, dữ liệu sống ở đúng địa chỉ luôn.

Khi nào move thực sự tốn?

Khi compiler không thể tối ưu — vd `Vec<i32>` với 1000 phần tử trong vòng lặp. Lúc đó move = **memcpy** 3 word (24 byte) — vẫn rất nhanh, nhưng có cost.

So với C++

C++11 có move semantics tương tự, nhưng **không bắt buộc**:

```
std::string s1 = "hi";
std::string s2 = s1;           // COPY (đắt!)
std::string s3 = std::move(s1); // MOVE (rẻ)
```

Rust: **mọi assignment đều là move mặc định** → ép programmer tránh copy ngầm.

29. Borrowing & Cache friendliness

Borrow checker khuyến khích pattern thân thiện cache.

Vì sao?

```
// Pattern 1: borrow
fn process(v: &Vec<i32>) {
    for x in v {
        // ...
    }
}

// Pattern 2: clone
fn process(v: Vec<i32>) { // forced clone trước khi pass
    // ...
}
```

Pattern 1: chỉ 1 vùng heap, được CPU prefetch tốt → cache hit cao.

Pattern 2: clone tạo vùng heap mới ở **xa** (allocator có thể chọn page khác) → cache miss khi load.

Slice — công cụ tối ưu cho cache

```
fn sum(arr: &[i32]) -> i32 {
    arr.iter().sum()
}
```

`&[i32]` = fat pointer (ptr + len) trở vào vùng liền kề. CPU prefetch tuần tự → siêu nhanh.

So với `LinkedList<i32>` (mỗi node alloc riêng, rải rác trên heap):



liền kề, prefetch tốt

cách nhau xa, cache miss liên tục

→ Rust idiomatic = dùng Vec/slice = cache-friendly mặc định.

Aliasing

Aliasing = 2 pointer trỏ cùng vùng nhớ. Aliasing là kẻ thù của optimizer.

```
// C – compiler GIẢ ĐỊNH a và b có thể overlap
void add(int* a, int* b, int* c) {
    *c = *a + *b;    // Có thể c == a → ghi a
    *c = *a + *b;    // Phải load lại *a (vì có thể đã thay đổi!)
}
```

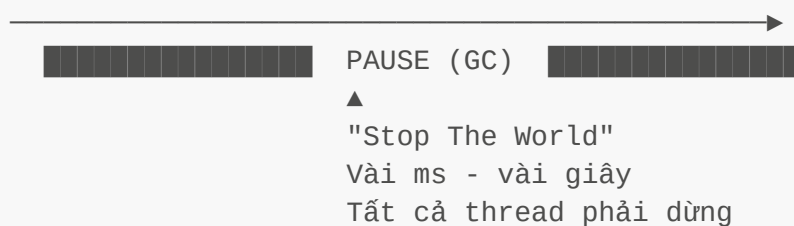
```
// Rust – borrow rules CẤM aliasing
fn add(a: &i32, b: &i32, c: &mut i32) {
    *c = *a + *b;    // Compiler CHẮC CHẮN c không phải a/b
    *c = *a + *b;    // Có thể optimize: load *a 1 lần
}
```

→ Borrow rules cho phép Rust optimize **mạnh hơn C** trong nhiều trường hợp.

30. Drop deterministic = không GC pause

GC pause là gì?

Java program timeline:



Trong các app real-time (game, trading, audio) → catastrophe.

Rust drop pattern

Rust program timeline:



```
(tại đúng (deterministic
} cuối scope) như C)
```

Mỗi drop **xảy ra tại đúng 1 điểm trong code**, được compiler biết trước. Không có pause khó dự đoán.

Hệ quả ưu việt cho:

- Game (16 ms/frame, không được pause > 1 ms)
- HFT trading (microsecond matters)
- Audio (44.1 kHz = ~22 µs/sample)
- Embedded (RAM hạn chế, không cần overhead GC)
- Kernel (không GC nào tồn tại trong kernel)
- WASM (binary nhỏ – không cần runtime GC)

31. Aliasing — vũ khí tối ưu

LLVM (backend Rust) có annotation `noalias`. C có `restrict` keyword nhưng ít ai dùng. Rust **tự động** áp dụng `noalias` cho mọi `&mut T`.

Ví dụ thực tế

```
fn modify(x: &mut i32, y: &i32) {
    *x = *y + 1;
    *x = *y + 1;          // compiler có thể loại bỏ 1 lệnh
}
```

Vì borrow rules đảm bảo `x` và `y` **không trùng** → đọc `*y` 1 lần là đủ.

Lịch sử: LLVM `noalias` bug

Khoảng 2015-2020, Rust phải tắt `noalias` cho `&mut T` vì gặp bug trong LLVM. Rồi bật lại từng phần. Đây là minh chứng: optimization tận dụng borrow rules **mạnh đến mức** LLVM chưa kịp xử lý đúng.

→ Khi LLVM stable hơn, Rust thậm chí có thể tối ưu **hơn nữa**.

TẦNG 9 — PATTERNS NÂNG CAO

32. Type-state pattern

Dùng type system để bắt lỗi logic tại compile time.

Ví dụ: HTTP request builder

```

struct Request<State> {
    url: String,
    body: Option<String>,
    _state: PhantomData<State>,
}

struct NoUrl;
struct HasUrl;

impl Request<NoUrl> {
    fn new() -> Self { ... }
    fn url(self, url: String) -> Request<HasUrl> { ... }
}

impl Request<HasUrl> {
    fn send(self) -> Response { ... }
}

```

```

Request::new()
    .send();           // ✗ compile error: NoUrl không có .send

Request::new()
    .url("...")
    .send();           // ✓

```

→ Compiler **bắt buộc** đi qua state. Không thể send khi chưa set URL.

Mối liên hệ ownership

Type state thường dùng **move semantics**: **self** consume → trả về **Self<NewState>**. Sau khi gọi **.url(...)**, state cũ "biến mất" (move), state mới sinh ra.

```

Request<NoUrl>
  | .url(...) consume self
  ▼
Request<HasUrl>
  | .send() consume self
  ▼
Response

```

→ Không có cách nào "lùi lại" — state machine compile-time.

33. PhantomData — lifetime ảo

PhantomData<T> chiếm 0 byte runtime, nhưng nói với compiler: "Tôi giả vờ chứa T."

Dùng để gì?

1. Generic không dùng thực sự (như type-state ở trên)

2. Lifetime ràng buộc cho FFI

```
struct CRef<'a, T> {
    ptr: *const T,
    _marker: PhantomData<'a T>,
}
```

*const T là raw pointer — không có lifetime. PhantomData thêm &'a T để compiler **giả vờ** CRef chứa &'a T → áp dụng lifetime check.

3. Variance

Phức tạp — về cách Foo<&'a T> có quan hệ con-cha trong lifetime. Để khi khác.

34. Self-referential structs — vì sao khó

Struct có field trở vào field khác **của chính nó** — Rust cấm trực tiếp.

```
struct Bad {
    data: String,
    ptr: &str,    // trở vào data - ❌
}
```

Vì sao cấm? Vì **move** sẽ làm dangling:

```
Trước move:
Bad { data: →"hi", ptr: →"h" } @ địa chỉ 0x1000

Sau move sang địa chỉ 0x2000:
Bad { data: →"hi" (cùng heap), ptr: →0x1000.h ❌ DANGLING }
```

Vì **data** move theo struct, nhưng **ptr** lưu địa chỉ **tuyệt đối** của field cũ → khi struct move, ptr trở vào địa chỉ rỗng.

Giải pháp

1. **Đánh dấu Pin** — cấm move (xem mục 35).

2. **Dùng ouroboros crate** — macro tạo unsafe code đúng cách.

3. **Refactor**: tách thành 2 struct.

35. Pin — neo cố định

`Pin<P>` là wrapper quanh pointer P, **cấm di chuyển** giá trị nó trỏ tới.

```
let mut x = 5;
let mut p = Pin::new(&mut x);
// *p = 10;    // OK
// std::mem::swap(p, ...); // ❌
```

Vì sao tồn tại Pin?

Async Rust dùng nhiều! `async fn` được dịch thành state machine có thể **chứa &mut self-reference** vào field khác.

```
async fn foo() {
    let data = String::from("hi");
    let r = &data;           // r tham chiếu data
    some_await.await;        // có thể bị move TẠI ĐÂY!
    println!("{}", r);
}
```

Nếu state machine move giữa các await → `r` dangling. → Pin **cấm move** state machine sau khi đã polled.

Sơ đồ Pin

```
Pin<&mut StateMachine>
  |
  |  cấm move
  ▼
StateMachine {
    data: String,
    r: &data,    ← OK, vì Pin neo machine tại địa chỉ cố định
}
```

→ Đa số code Rust không cần dùng Pin. Chỉ async runtime và unsafe code mới cần.

TẦNG 10 — BẦY THƯỜNG GẶP

36. Common errors & cách đọc

Lỗi 1: `cannot move out of borrowed content`

```
fn first(v: &Vec<String>) -> String {
    v[0]    // ✗
}
```

Vì sao? `v[0]` muốn move String ra. Nhưng `v` chỉ là `&Vec` → không được phép xé String ra.

Sửa: clone hoặc trả về reference

```
fn first(v: &Vec<String>) -> &String {
    &v[0]
}
// hoặc
fn first(v: &Vec<String>) -> String {
    v[0].clone()
}
```

Lỗi 2: cannot borrow x as mutable because it is also borrowed as immutable

```
let mut s = String::from("hi");
let r1 = &s;
s.push_str("!");           // ✗
println!("{}", r1);
```

Vì `&mut` và `&T` xung đột. Sửa: dùng `r1` trước push, hoặc tách scope.

```
let mut s = String::from("hi");
{
    let r1 = &s;
    println!("{}", r1);
}
s.push_str("!");           // ✓
```

Lỗi 3: borrowed value does not live long enough

```
let r;
{
    let x = 5;
    r = &x;           // ✗
}
println!("{}", r);
```

→ `x` chết khi `}` đóng. `r` cố sống lâu hơn → cấm.

Lỗi 4: `cannot return reference to local variable`

```
fn make() -> &String {
    let s = String::from("hi");
    &s           // ✗
}
```

→ `s` chết khi hàm return. Sửa: return owned String.

```
fn make() -> String {
    String::from("hi")
}
```

Lỗi 5: `use of moved value`

```
let s = String::from("hi");
takes(s);
println!("{}", s);    // ✗
```

Sửa: borrow thay vì move, hoặc clone.

```
let s = String::from("hi");
takes(&s);           // borrow
println!("{}", s);   // ✓
```

Đọc lỗi như đọc bản đồ

Mọi lỗi borrow checker đều chỉ rõ:

- Borrow ở đâu (line nào)
- Loại gì (mut / immut)
- Conflict với cái nào
- Suggest fix

```
error[E0502]: cannot borrow `s` as mutable...
--> src/main.rs:4:5
  |
3 |     let r = &s;
  |             -- immutable borrow occurs here
4 |     s.push_str("!");
  |     ^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
```

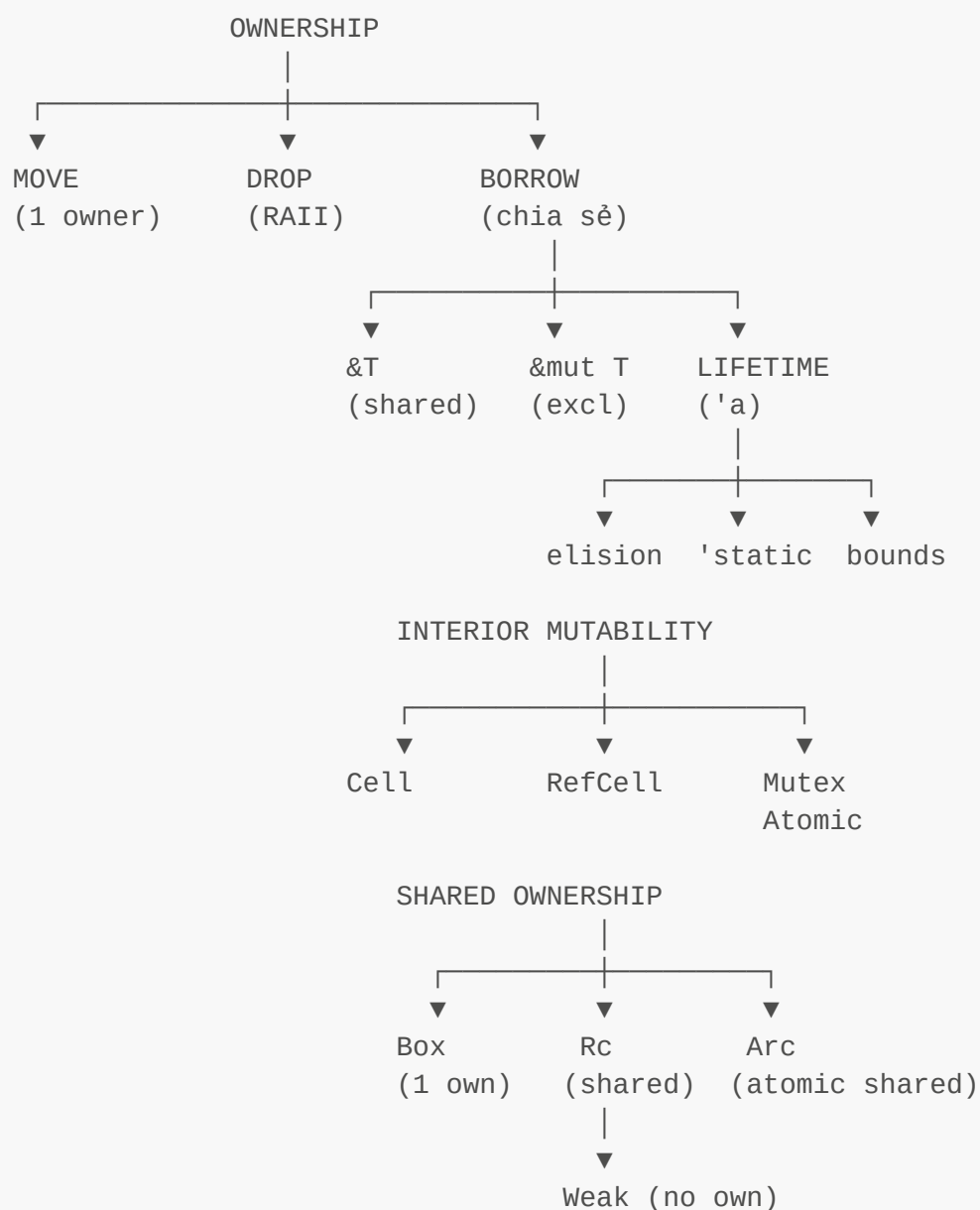


```
5 |     println!("{}", r);
  |     - immutable borrow later used here
```

→ Đọc top-down, tìm 3 dòng có dấu - và ^ — đó là 3 "thời điểm" Rust phát hiện vi phạm.

KẾT LUẬN

Bản đồ tư duy



Nguyên lý cốt lõi để ghi nhớ

- | | |
|--------------------|--|
| 1. AI LÀ OWNER? | → Theo dõi "ai chịu trách nhiệm free" |
| 2. AI ĐANG BORROW? | → Theo dõi "ai đang xem/sửa" |
| 3. BAO LÂU? | → Theo dõi lifetime – không sống lâu hơn dữ liệu |

4. CÓ ALIAS &mut KHÔNG? → Không bao giờ! Nếu cần, dùng interior mutability
5. SHARED HAY EXCLUSIVE? → &T vs &mut T tương ứng RwLock readers vs writer

Quan hệ với Memory Model

Ownership	↔ heap allocator deterministic (no GC)
Move	↔ zero-copy semantic (chỉ chuyển ptr trên stack)
Borrow rules	↔ noalias → optimizer mạnh + cache friendly
Drop	↔ RAII, predictable cleanup
Lifetime	↔ static analysis của reference lifetimes
Interior mut	↔ wrapper trên UnsafeCell (compiler đặc biệt)
Rc/Arc	↔ reference count thay GC, không cycle detection
Pin	↔ async self-referential, neo địa chỉ

Lộ trình tiếp theo nên học

1. **Trait & Generic** — đa hình + zero-cost abstraction
2. **Closure & Fn/FnMut/FnOnce** — gắn liền với borrow rules
3. **Iterator** — lazy evaluation, monad-like composition
4. **Async/Await** — Future trait, runtime (tokio)
5. **Smart pointers nâng cao** — Cow, ManuallyDrop, MaybeUninit
6. **Unsafe Rust** — raw pointer, transmute, FFI

Mỗi cái đều dựa trên nền **ownership** đã học ở đây.

Đọc song song với [memory-model.md](#) và [memory-model-visual.md](#) — vì borrow checker là **mô hình hoá memory safety** trên nền memory layout. Hiểu memory model → hiểu vì sao quy tắc borrow tồn tại.